

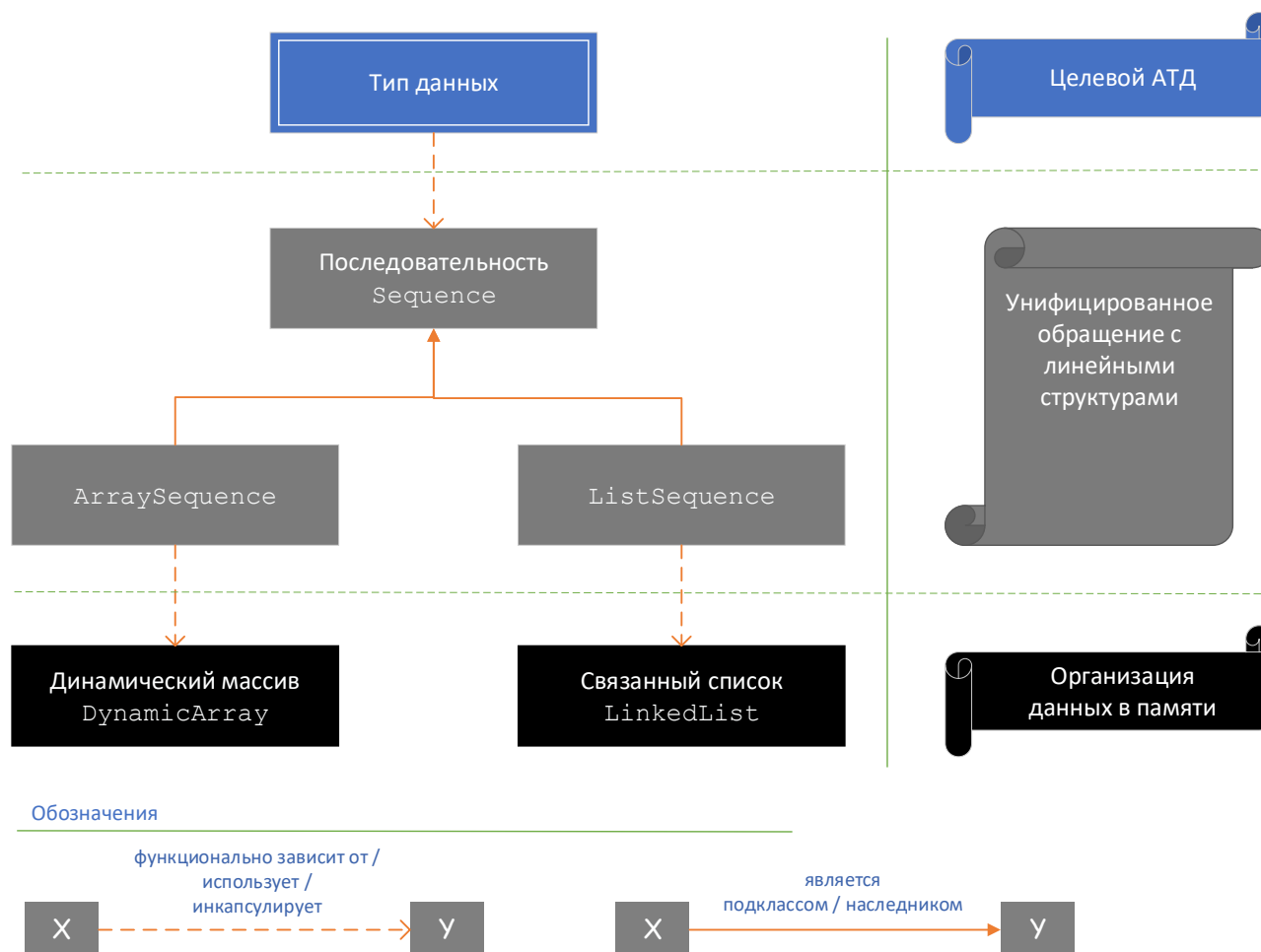
Лабораторная работа №2

по курсу информатики, 2 семестр

Варианты заданий

1. Постановка задачи

Написать на языке C++ систему для линейной организации с помощью нескольких полиморфных абстрактных типов данных – Динамический массив, Связанный список и Последовательность. При этом АТД Последовательность представляет собой абстракцию, предоставляющую унифицированный доступ к двум другим (см. рис. ниже). Реализуемая система должна предоставлять расширенный функционал для использования при реализации АТД и алгоритмов в последующих ЛР во 2-м и 3-м семестрах.



Следует предусмотреть средства для оценки производительности полученной реализации.

Минимальные требования к программе. В составе лабораторной работы должны быть реализованы:

- АТД динамический массив,
- АТД линейный связанный список,
- АТД последовательность,

-
- оболочка с UI для тестирования полученной системы объектов,
 - АТД последовательность должен быть реализован в двух вариантах: как изменяемая (mutable) и как неизменяемая (immutable) структура,
 - реализовать специальный вариант последовательности – битовую последовательность BitSequence, которая должна реализовывать интерфейс Sequence<Bit>, где Bit – абстракция, позволяющая представить битовое значение.

Для реализации необходимо использовать возможности ООП и шаблонов C++ (templates) – классов и функций. Во всех реализованных функциях необходимо обрабатывать случаи некорректных значений входных параметров – как правило, в таких случаях следует выбрасывать исключения.

BitSequence, помимо обычных для последовательностей операций, должен реализовывать операции, специфичные для битовых массивов – побитовые AND, OR, XOR, NOT. При реализации необходимо учесть потребность в использовании этого АТД как битовой маски.

Все реализованные классы и основные алгоритмы необходимо покрыть (модульными) тестами. Реализацию следует оснастить пользовательским интерфейсом (консольным) для проверки корректности реализации.

Изменяемые (mutable) и неизменяемые (immutable) структуры данных. Отличительная особенность неизменяемых структур по сравнению с изменяемыми состоит в том, что, реализуя схожий, а зачастую – идентичный, интерфейс и набор операций, – неизменяемые структуры являются «чистыми» (pure), а точнее, чистыми, или функционально чистыми, являются их методы. Термин «функциональная чистота», или просто «чистота», применяется, чтобы обозначить что некоторый код ведет себя как «чисто математическая» сущность: результат работы кода полностью определяется исходными данными для него, которые задаются явно. Это означает, что:

- работа методов не зависит от внешней среды (по крайней мере, отсутствуют *неявные* зависимости¹);
- в результате работы не меняется окружение (среда), и, наоборот, изменение окружения в процессе выполнения методов не влияет на их работу²;
- результат выполнения методов может зависеть, и как правило зависит, от внутреннего состояния объекта, но не меняет это состояние³.

Обычно свойство неизменяемости достигается тем, что операции, которые в ином случае меняли бы состояние объекта, вместо этого создают копию исходного объекта, вносят изменения в эту копию и в качестве результат возвращают именно эту копию, оставляя исходный объект без изменений.

В данной работе необходимо выполнить и сравнить работу как изменяемой, так и неизменяемой последовательности. Это необходимо сделать хотя бы для одного вариантов последовательности – ArraySequence или/и ListSequence.

Дополнительные требования к программе (на повышение оценки).

- Перегрузка операторов (например, оператора [] для обращения/задания значения элемента по индексу).
- Поддержка комплекса операций map-reduce:

¹ Т.е. работает принцип абстрагирования зависимостей [от внешней среды], т.е. все зависимости от чего-либо, лежащего за пределами данного объекта, объявляются явно.

² Принцип изоляции, или замкнутости.

³ В контексте ООП состоянием объекта уместно называть совокупность значений его полей (в т.ч. и унаследованных). Однако в контексте неизменяемых структур сам термин «состояние» не особенно применим.

- from
 - map,
 - flatmap
 - reduce/fold,
 - find/where,
 - zip и unzip.
- Поддержка итераторов: поддержка интерфейса `IEnumerable<T>`, реализация `IEnumerator<T>`.
 - Реализация типа `Option<T>` и try-семантики. Try-семантика состоит в том, что «поисковые» методы типа `find`, `first` и некоторые другие снабжаются вариантами, которые отличаются следующим:
 - имя начинается с префикса Try-
 - возвращаемый тип не `T`, а `Option<T>`
 - если элемент не удастся найти, исключение не выбрасывается, а возвращается значение `Option<T>.None`.
 - Реализовать функцию `Split`, которая данную последовательность элементов разобьет на отдельные фрагменты. Границы между фрагментами – это элементы, удовлетворяющие условию, которое передается как параметр.
 - Реализовать функцию `Slice`, которая работает примерно так же, как в JavaScript: на вход подается индекс i , количество элементов N и, возможно, пустая, последовательность элементов s . Если индекс отрицательный, отсчет ведется с конца последовательности, а если он по модулю больше длины последовательности, следует выбросить исключение. Функция удаляет N элементов начиная с позиции i . При этом вместо удаленных элементов вставляются элементы из последовательности s (очевидно, если она не пустая).

```
Sequence<int> * seq = new ArraySequence(new [] {1,2,3,4,5});
```

```
Seq->Slice(1,2, new ArraySequence(new [] {9,10})); // == {1,9,10,4,5}
```

Перечисленные возможности могут быть реализованы как на уровне только `Sequence`, так «протянуты» через все уровни абстракции.

2. Требования к структурам данных

2.1. Класс `DynamicArray`

template <class T> class DynamicArray	
Создание объекта	
DynamicArray(T* items, int count);	Копировать элементы из переданного массива
DynamicArray(int size);	Создать массив заданной длины
DynamicArray(DynamicArray<T> & dynamicArray const);	Копирующий конструктор

Декомпозиция	
T Get(int index); Может выбрасывать исключения: IndexOutOfRangeException (если индекс отрицательный, больше/равен числу элементов или указывает на не заданный элемент)	Получить элемент по индексу.
int GetSize();	Получить размер массива
Операции	
void Set(int index, T value);	Задать элемент по индексу Может выбросить IndexOutOfRangeException
void Resize(int newSize);	Изменить размер массива. Если размер увеличивается, все элементы копируются в начало новой памяти. Если уменьшается – элементы, которые не помещаются, отбрасываются.

2.2. Класс LinkedList

template <class T> class LinkedList	
Создание объекта	
LinkedList (T* items, int count);	Копировать элементы из переданного массива
LinkedList ();	Создать пустой список
LinkedList (LinkedList <T> & list const);	Копирующий конструктор
Декомпозиция	
T GetFirst(); Может выбрасывать исключения: IndexOutOfRangeException (если список пуст)	Получить первый элемент в списке
T GetLast(); Может выбрасывать исключения: IndexOutOfRangeException (если список пуст)	Получить последний элемент в списке
T Get(int index);	Получить элемент по индексу.

Может выбрасывать исключения: — <code>IndexOutOfRangeException</code> (если индекс отрицательный или больше/равен числу элементов)	
<code>LinkedList<T>* GetSubList(int startIndex, int endIndex);</code> Может выбрасывать исключения: — <code>IndexOutOfRangeException</code> (если хотя бы один из индексов отрицательный или больше/равен числу элементов)	Получить список из всех элементов, начиная с <code>startIndex</code> и заканчивая <code>endIndex</code> .
<code>int GetLength();</code>	Получить длину списка
Операции	
<code>void Append(T item);</code>	Добавляет элемент в конец списка
<code>void Prepend(T item);</code>	Добавляет элемент в начало списка
<code>void InsertAt(T item, int index);</code> Может выбрасывать исключения: — <code>IndexOutOfRangeException</code> (если индекс отрицательный или больше/равен числу элементов)	Вставляет элемент в заданную позицию
<code>LinkedList<T>* Concat(LinkedList<T> *list);</code>	Сцепляет два списка

2.3. Класс Sequence

<code>template <class T> class Sequence</code> <code>template <class T> class ArraySequence : Sequence<T></code> <code>template <class T> class ListSequence : Sequence<T></code>	
Создание объекта	
<code>ArraySequence (T* items, int count);</code> <code>ListSequence (T* items, int count);</code>	Копировать элементы из переданного массива
<code>ArraySequence ();</code> <code>ListSequence ();</code>	Создать пустой список
<code>ArraySequence (LinkedList <T> & list const);</code> <code>ListSequence (LinkedList <T> & list const);</code>	Копирующий конструктор

Декомпозиция	
T GetFirst(); Может выбрасывать исключения: — IndexOutOfRangeException (если список пуст)	Получить первый элемент в списке
T GetLast(); Может выбрасывать исключения: — IndexOutOfRangeException (если список пуст)	Получить последний элемент в списке
T Get(int index); Может выбрасывать исключения: — IndexOutOfRangeException (если индекс отрицательный или больше/равен числу элементов)	Получить элемент по индексу.
Sequence<T>* GetSubsequence(int startIndex, int endIndex); ArraySequence<T>* GetSubsequence(int startIndex, int endIndex); ListSequence<T>* GetSubsequence(int startIndex, int endIndex); Может выбрасывать исключения: — IndexOutOfRangeException (если хотя бы один из индексов отрицательный или больше/равен числу элементов)	Получить список из всех элементов, начиная с startIndex и заканчивая endIndex.
int GetLength();	Получить длину списка
Операции	
Sequence<T>* Append(T item); ArraySequence<T>* Append(T item); ListSequence<T>* Append(T item);	Добавляет элемент в конец списка
Sequence<T>* Prepend(T item); ArraySequence<T>* Prepend(T item); ListSequence<T>* Prepend(T item);	Добавляет элемент в начало списка
Sequence<T>* InsertAt(T item, int index); ArraySequence<T>* InsertAt(T item, int index); ListSequence<T>* InsertAt(T item, int index); Может выбрасывать исключения: — IndexOutOfRangeException (если индекс отрицательный или больше/равен числу элементов)	Вставляет элемент в заданную позицию
Sequence <T>* Concat(Sequence <T> *list);	Сцепляет два списка

Sequence <T>* Map(T2 (*)(T)); Sequence <T>* Where(bool (*)(T)); Sequence <T>* Reduce(T2 (*)(T2,T)); Option<T> GetFirst(bool (*)(T) = 0); Option<T> GetLast(bool (*)(T) = 0);	Базовый комплект операций map-reduce.
IEnumerator<T> GetEnumerator();	Возвращает итератор, подходящий для конкретной последовательности.

Кроме того, здесь же оптимально реализовывать операции из серии map-reduce: map, reduce, zip, unzip, where (и др.).

Примерная схема реализации – на примере класса ArraySequence:

```
// Упрощенная реализация, без «умного» управления буфером.
template <class T>
class ArraySequence : Sequence<T>
{
private:
    DynamicArray<T>* items;
    //...
public:
    //...
    int GetLength()
    {
        return this->items->GetSize();
    }
    //...
    void Append(T item)
    {
        this->items->Resize(this->items->GetSize()+1);
        this->items->Set(this->items->GetSize()-1, item);
    }
};
```

Т.е. для реализации используется инкапсуляция DynamicArray и делегирование ему большей части работы.

Реализация mutable- и immutable-вариантов. По сути, с точки зрения кода, различие между mutable/immutable вариантами большинства структур состоит лишь в том, что в изменяемом варианте необходимые изменения вносятся в текущий объект, а в случае неизменяемой структуры – предварительно создается копия, с которой производятся те же действия, которые до этого производились над исходным объектом. Один из наиболее очевидных и прямолинейных вариантов реализации обеих вариаций состоит в вынесении основной части кода в базовый класс. Например:

```

Sequence<T>* ArraySequence::Append(T item)
{
    return Instance()->AppendInternal(item);
}
virtual Sequence<T>* ArraySequence::Instance() = 0;
virtual Sequence<T>* MutableArraySequence::Instance() override
{
    return this;
}
virtual Sequence<T>* ImmutableArraySequence::Instance() override
{
    return this->Clone();
}

```

Комплекс map-reduce. Если $l = [a_1, \dots, a_n]$ – некоторый список элементов типа T , а $f: T \rightarrow T$, то:

$$\text{map}(f, l) \mapsto [f(a_1), \dots, f(a_n)]$$

Если, при тех же соглашениях, $h: T \rightarrow \text{Bool}$ – некоторая функция, возвращающая булево значение, то результатом $\text{where}(h, l)$ будет новый список l' , такой что: $a'_i \in l' \Leftrightarrow h(a'_i) = \text{true}$. Т.е. where фильтрует значения из списка l с помощью функции-фильтра h .

На практике при решении задач, использующих map , иногда требуется знать не только значение элемента, но и информация о списке в целом, например, положение данного элемента относительно других. Например, при подсчете количества инверсий в списке полезно знать индекс каждого элемента, чтобы не вычислять его отдельно. В этом случае функция, отвечающая за поэлементное преобразование, принимает дополнительный аргумент – индекс элемента в списке: $f: T_1 \times \text{Int} \rightarrow T_2$.

Функция reduce работает несколько иначе: «сворачивает» список в одно значение по заданному правилу $f: T \times T \rightarrow T$:

$$\text{reduce}(f, l, c) \mapsto f \left(a_n, \left(f \left(a_{n-1}, \left(\dots f \left(a_2, (f(a_1 c)) \right) \right) \right) \right) \right)$$

где c – константа, «стартовое» значение. Например, $l = [1, 2, 3]$, $f(x_1, x_2) = 2x_1 + 3x_2$, тогда:

$$\begin{aligned}
 \text{reduce}(f, [1, 2, 3], 4) &= f(3, f(2, f(1, 4))) = \\
 &= 2 \cdot 3 + 3(2 \cdot 2 + 3(2 \cdot 1 + 3 \cdot 4)) = \\
 &= 2 \cdot 3 + 3(2 \cdot 2 + 3 \cdot 14) = 2 \cdot 3 + 3 \cdot 42 = 132
 \end{aligned}$$

Пара функций zip и unzip позволяет «сцеплять вдоль» несколько списков и разъединять их.

$$\begin{aligned}
 \text{zip}: \text{List}\langle T_1 \rangle \times \text{List}\langle T_2 \rangle &\rightarrow \text{List}\langle \text{Tuple}\langle T_1, T_2 \rangle \rangle \\
 \text{zip3}: \text{List}\langle T_1 \rangle \times \text{List}\langle T_2 \rangle \times \text{List}\langle T_3 \rangle &\rightarrow \text{List}\langle \text{Tuple}\langle T_1, T_2, T_3 \rangle \rangle \\
 \text{zipN}: \text{List}\langle \text{List}\langle T \rangle \rangle &\rightarrow \text{List}\langle \text{List}\langle T \rangle \rangle
 \end{aligned}$$

Примеры:

`zip([1,2,3],["a","b","c","d",]) = [(1,"a"), (2,"b"), (3,"c")]`

`zipN([1,2,3], [1,2,3], [1,2,3]) = [[1,1], [2,2], [3,3]]`

Функция `unzip` является обратной к `zip`: «распаковывает» список кортежей в кортеж списков.

Указатели на функции. Ниже – минимальный пример, как создать «список функций»:

```
const int array_length = 3;
int(**f)(int) = malloc(array_length * sizeof(int(*) (int)));
f[0] = &inc1;
f[1] = &inc2;
f[2] = &inc3;
for (int index = 0; index < length; index++)
    printf("%i ", f[index](0));
// Вывод: 1 2 3
```

(Задание должно включать пункты общей «стоимостью» не менее 9 баллов.)

Таблица 2. Содержание вариантов заданий			
Код	Задача	Пояснения	Баллы
Б-1.	Связанный список		
Б-2.	Динамический массив		
Б-3.	Последовательность		
Б-3.1.	ListSequence		
Б 3.1.1.	Варианты MutableListSequence и ImmutableListSequence		2
Б 3.1.2.	Реализация Builder'a		
Б-3.2.	ArraySequence		
Б 3.2.1.	Варианты MutableListSequence и ImmutableListSequence		2
Б 3.2.2.	Реализация Builder'a		
Б-3.3.	AdaptiveSequence		3
Б-3.4.	SegmentedList		5
М-1.	ICollection<T>	Основные методы – T Get(size_t) –size_t GetCount() –копирующий конструктор	2
М-2.	Map-Reduce		
М-2.1.	Дополнительные	Zip/unzip, Skip, Split, S(p)lice, FlatMap, генераторы (напр. Range)	2
М-2.2.	Map с контекстом (индекс, индекс + двусторонний итератор)		1
М-2.3.	Реализация монад		

М-2.3.1.	с использованием ResultInfo		3
М-2.3.2.	с использованием шаблонов		3
М-3.	Использовать IGroup/IRing	–	4
М-4.	Перегрузка операторов ⁸⁾		2
М-5.		–	
М-6.		–	
М-7.		–	
С помощью map-reduce, минимизируя объем вычислений, вычислить:			
П-1.	(min, max, avg)	возвращает тройку за 1 проход	1
П-2.	медиана		1 или 2
П-3.	кол-во перестановок		1 или 2
П-4.	для каждого элемента – множество (последовательность) элементов, предшествующих ему в данной посл-ти и одновременно меньших, чем данный		1
П-5.	список всех префиксов/постфиксов		1
П-6.	для каждого элемента $a_i: (a_{i-1} + a_i + a_{i+1})/3$		2
П-7.	для каждого элемента $a_i: \sqrt{\sigma^2 - a_i^2}$, где σ – среднее квадратичное		2
П-8.	список элементов вида $a_i + a_{n-i}$ (поэлементная сумма списка с его «отражением»)		1
П-9.			
П-10.			

3. Критерии оценки

Оценка лабораторной работы осуществляется по нескольким показателям, по каждому из которых предусматривается 2 категории критериев:

1. базовые требования, без соблюдения которых работа считается незачтенной, а по показателю выставляется 0 баллов;
2. количественные критерии, по которым оценивается исходный код и результирующая программа как метрики освоения студентом методик, приемов, идей, технологий, инструментов программной разработки.

1.	Качество программного кода:		0-7 баллов
1.1.	базовые требования	– реализация полиморфного АД – систематическое следование выбранному стилю оформления кода, информативность имен, «читабельность» кода – отсутствие утечек памяти – логическая обоснованность всех принятых	

		решений – корректное разделение ответственностей (в частности, отсутствие печати в консоль в структурах данных и предметной логике) – реализация основных операций map-reduce	
1.2.	количественные критерии	– структурированность (декомпозиция сложных функций на более простые, гранулированность кода, только явные зависимости) (0-3) – качество предложенных API (полнота, удобство использования исходя из зафиксированных сценариев) (0-4)	
2.	Качество тестов		0-4 баллов
2.1.	базовые требования	– покрытие тестами основных участков предметной логики – охват пограничных ситуаций – охват некорректных ситуаций и сценариев	
2.2.	количественные критерии	– читаемость – степень покрытия	
3.	Обработка ошибок		0-4 баллов
3.1.	базовые требования	– обработка некорректных данных – гарантия, «программа не умрет молча» – систематическая работа с исключениями	
3.2.	количественные критерии	– Простота идентификации ошибки – Простота локализации ошибки – Простота и полнота расшифровки содержания/причины ошибки	
		Итого	0-15 баллов
<i>Бонусные задачи</i>			
4.	Качество пользовательского интерфейса:		0-5 баллов
4.1.	базовые требования	– обеспечение пользователю возможности выполнять основные задачи: создание объектов, проверка корректности выполнения их различных операций – предоставление документированного набора команд	
4.2.	количественные критерии	– предоставляемые им возможности – наличие ручного/автоматического ввода исходных данных – настройка параметров для автоматического режима – отображение исходных данных и промежуточных и конечных результатов и др.	
5.	Оригинальность реализации	– наличие возможностей в API из разряда «quality of life» – продвинутая работа с шаблонами – продвинутые решения по UI	0-5 баллов

		– и др.	
--	--	---------	--

Для получения зачета за выполнения лабораторной работы необходимо соблюдение всех перечисленных условий:

- основные структуры данных и алгоритмы должны быть реализованы
- должны быть выполнены требования по всем базовым критериям
- по всем количественным критериям 1-3 оценка должна быть >0
- общий балл по всем критериям должен быть $\geq 60\%$ от основной оценки (т.е. ≥ 9)